Question 2 - Terraform infrastructure as code

Question 2: Provision scalable infrastructure using Terraform (Infrastructure as Code): write configuration files, initialize, plan, and apply the infrastructure.

All commands below were run on the lab machine and their output is copied verbatim from the recorded logs in `evidence/logs/`. Repository: https://github.com/ksamirdev/ensp461-devops-lab3.

Configuration

infra/main.tf

```
terraform {
  required_version = ">= 1.9"
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.6"
    }
  }
}

provider "docker" {}

variable "replicas" {
  description = "Number of text-stats containers"
  type        = number
  default     = 2

  validation {
    condition     = var.replicas >= 1 && var.replicas <= 10
    error_message = "replicas must be between 1 and 10 (ports 8310-8319 are reserved)."
  }
}

variable "release" {
  description = "Release reported by the service"
  type        = string
  default     = "3.0.0"
}

resource "docker_network" "lab" {
  name = "lab3-tf"
}

# Built beforehand with `docker build`; Terraform only provisions from it
data "docker_image" "text_stats" {
  name = "text-stats:${var.release}"
}

resource "docker_container" "text_stats" {
  count = var.replicas
  name  = "text-stats-${count.index + 1}"
  image = data.docker_image.text_stats.id
  env   = ["RELEASE=${var.release}"]

  networks_advanced {
    name = docker_network.lab.name
  }
  ports {
```

```
    internal = 5000
    external = 8310 + count.index
    ip       = "127.0.0.1"
  }
  restart = "unless-stopped"

  # The Docker daemon injects default log options; without this every plan replaces the containers
  lifecycle {
    ignore_changes = [log_opts]
  }
}

output "urls" {
  value = [for c in docker_container.text_stats : "http://localhost:${c.ports[0].external}"]
}
```

Init, validate, plan, apply

```
$ docker build -q -t text-stats:3.0.0 .
sha256:fbe39bb66ceb30544eb6a4945879e8ecccd5dbaa87fdfea837045868ddf756b2
$ terraform version | head -1
Terraform v1.17.1
$ terraform init -no-color
Initializing the backend...

Initializing provider plugins...
- Reusing version 3.9.0 of kreuzwerker/docker from the dependency lock file
- Using previously-installed kreuzwerker/docker v3.9.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
$ terraform fmt -check && terraform validate -no-color
Success! The configuration is valid.
```

plan output (first lines; the full plan is in evidence/logs/q2-terraform.log)

```
$ terraform plan -no-color -out=tfplan
data.docker_image.text_stats: Reading...
data.docker_image.text_stats: Read complete after 0s [id=sha256:fbe39bb66ceb30544eb6a4945879e8ecccd5
dbaa87fdfea837045868ddf756b2]

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # docker_container.text_stats[0] will be created
  + resource "docker_container" "text_stats" {
      + attach                                      = false
      + bridge                                      = (known after apply)
      + command                                     = (known after apply)
...
```

```
$ terraform apply -no-color -auto-approve tfplan
...
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

```
Outputs:

urls = [
  "http://localhost:8310",
  "http://localhost:8311",
]
```

```
$ curl -s http://localhost:8310/healthz
```

Scaling from 2 to 4 replicas

```
$ terraform plan -no-color -var replicas=4 | tail -8
      + "http://localhost:8312",
      + "http://localhost:8313",
    ]

──────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.
$ terraform apply -no-color -auto-approve -var replicas=4 | tail -12
docker_container.text_stats[2]: Creation complete after 1s [id=410150966bf5f01ae75cf5ba273ca2a4155dc
45484390ee87ca29f80f3623e75]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

urls = [
  "http://localhost:8310",
  "http://localhost:8311",
  "http://localhost:8312",
  "http://localhost:8313",
]
$ curl -s http://localhost:8313/healthz
{"healthy":true,"release":"3.0.0"}
$ terraform state list
data.docker_image.text_stats
docker_container.text_stats[0]
docker_container.text_stats[1]
docker_container.text_stats[2]
docker_container.text_stats[3]
docker_network.lab
```

Scale back and drift check

```
$ terraform apply -no-color -auto-approve -var replicas=2 | tail -8
Apply complete! Resources: 0 added, 0 changed, 2 destroyed.

Outputs:

urls = [
  "http://localhost:8310",
  "http://localhost:8311",
]
$ terraform plan -no-color -detailed-exitcode | tail -2; echo exit=${PIPESTATUS[0]}
Terraform has compared your real infrastructure against your configuration
and found no differences, so no changes are needed.
exit=0
```

Observations

- Scaling is a single variable: `-var replicas=4` added exactly 2 containers (**2 added, 0 changed, 0 destroyed**), and each new container got the next free port automatically.
- `terraform plan -detailed-exitcode` returned **0** after the changes, so the real infrastructure matches the code exactly.
- My first attempt replaced *every* container on each scale. The plan showed `log_opts` "forces replacement": the Docker daemon adds default log options that are not in my config. Fixed with `lifecycle { ignore_changes = [log_opts] }`.
- Building the image inside Terraform hung on this engine, so the image is built with `docker build` first and Terraform reads it with a `docker_image` data source. Build and provisioning stay separate steps.
- The validation block rejects replica counts outside 1-10, so a typo cannot ask for hundreds of containers.